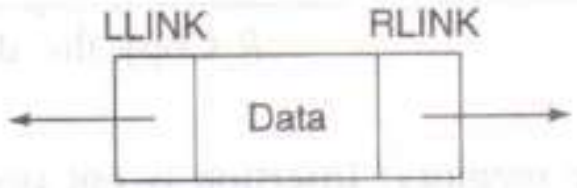
DOUBLY LINKED LIST

DOUBLY LINKED LIST

- The doubly linked list is a collection of nodes each of which consists of **three parts** namely the **data part**, **Llink pointer** and the **Rlink pointer**.
- The **data part** stores the **value of the element**, the **Llink pointer** has the **address of the previous node** and the **Rlink pointer** has the address of the **next node** in the list.

- In a **singly linked list one can move beginning from** the **header node to any node in one direction only**(from **left** to **right**). This is why a single linked list is also known as a **one - way list.**
- On the other hand , a doubly linked list is a **two-way list** because one **can move in either direction** , **either from left** to **right** or from **right** to **left.** ***(forward*** **or** ***reverse).***
- A structure of a node for a double linked list is represented as



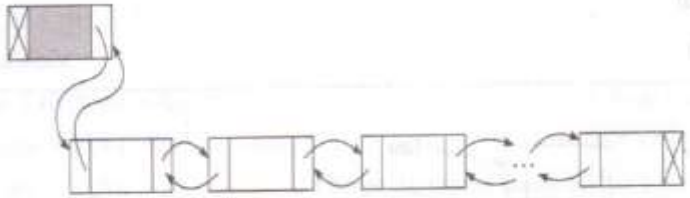


Figure 3.10 Structure of a node and a double linked list.

ADVANTAGES OF DOUBLY LINKED LIST

- **Traversal in either direction** becomes convenient.
- **Reduces time requirement for the program execution.**
- Doubly linked lists can be **used to represent other data structures.**
- The hierarchical structure of the **tree** can be easily represented using a doubly linked list.
- **Graphs** can be represented using doubly linked list.

DISADVANTAGES OF DOUBLY LINKED LIST

- **Each node requires extra space for storing the additional pointer.**
- While manipulating the lists, **extra care should be taken to manipulate both links**.

Doubly Linked List v/s Singly Linked List:

Singly Linked list	Doubly Linked list
1. Traversing is convenient in only one direction.	1. Traversing can be done in both directions.
2. While deleting a node, its predecessor is required and can be found only after traversing from the beginning of list.	2. While deleting a node, it's predecessor can be obtained using 'previouslink' of node. No need to traverse the list.
3. Occupies less memory.	3. Occupies more memory.
4. Program will be lengthy and need more time to design.	4. Using circular linked list with header, efficient and small programs can be written and hence design is easier.
5. Care is taken to modify only one link of a node.	5. Care is taken to modify both links of a node.

DOUBLY LINKED LIST OPERATIONS

1. Traversing (Display) all the elements of the list in either forward or backward direction.
2. Inserting a node into the list.
    i. insert an element at the beginning of the list
    ii. insert an element at the end of the list
    iii. insert an element at the specified position in the list
3. Deleting a node from the list
    i. Delete an element from the beginning of the list
    ii. Delete an element at the end of the list
    iii. Delete an element at the specified position in the list
4. Searching for an element in the list.
5. Count the number of elements.

TRAVERSING (DISPLAY) ALL THE ELEMENTS OF THE LIST IN THE FORWARD DIRECTION

```
void display_forward()
{
ptr=header->Rlink;
while(ptr!=NULL)
{
printf("\n %d",ptr->data);
ptr=ptr->Rlink;
}
}
```

TRAVERSING (DISPLAY) ALL THE ELEMENTS OF THE LIST IN THE BACKWARD DIRECTION

```
void display_backward()
{
ptr=trailer->Llink;
while(ptr!=NULL)
{
printf("\n %d",ptr->data);
ptr=ptr->Llink;
}
}
```

INSERTING A NODE INTO THE LIST.

There are various positions where a node can be inserted:

i. insert an element at the beginning of the list

ii. insert an element at the end of the list

iii. insert an element at the specified position in the list

INSERTING A NODE AT THE FRONT OF A DOUBLY LINKED LIST.

Algorithm InsertFront_DL

Input: X is the data content of the node to be inserted.

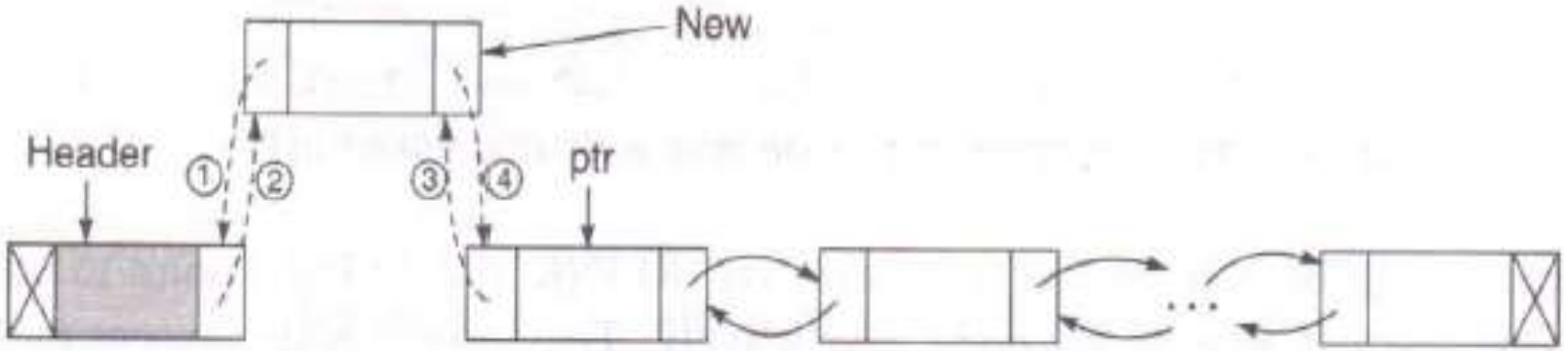
Output: A double linked list enriched with a node in the front containing data X.

Data structure: Double linked list structure whose pointer to the header node is the *HEADER*.

Steps:

1. ptr = HEADER→RLINK // Points to the first node
2. new = **GetNode**(NODE) // Avail a new node from the memory bank
3. **If** (new ≠ NULL) then // If new node is available
4. new→LLINK = HEADER // Newly inserted node points the header as 1 in Figure 3.11(a)
5. HEADER→RLINK = new // Header now points to then new node as 2 in Figure 3.11(a)
6. new→RLINK = ptr // See the change in pointer shown as 3 in Figure 3.11(a)
7. ptr→LLINK = new // See the change in pointer shown as 4 in Figure 3.11(a)

```
8.    new→DATA = X                    // Copy the data into the newly inserted node
9.  Else
10.     Print "Unable to allocate memory: Insertion is not possible"
11. EndIf
12. Stop
```



(a) Inserting a node in the front

INSERTING A NODE AT THE END OF A DOUBLY LINKED LIST.

Algorithm InsertEnd_DL

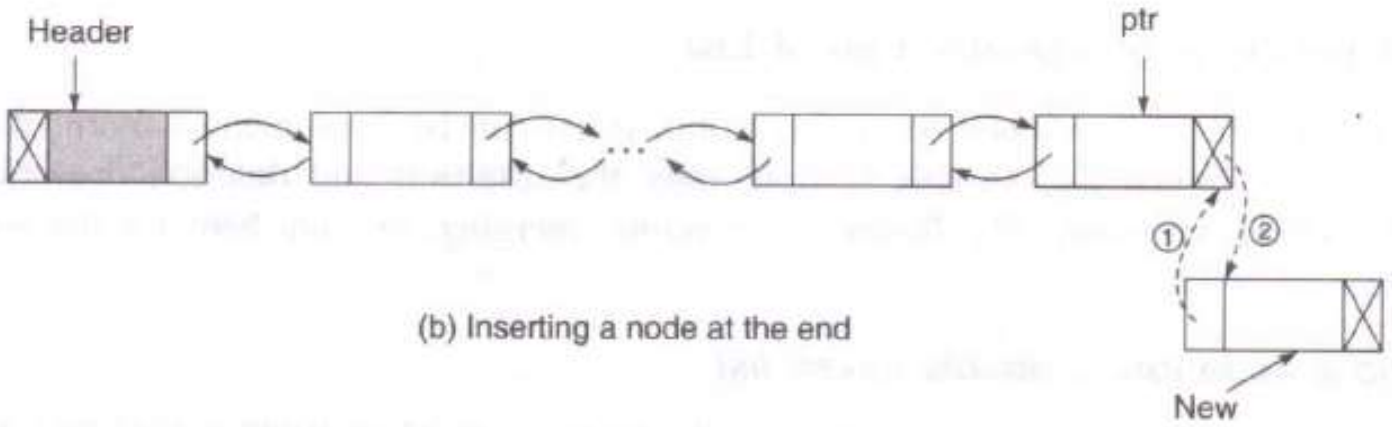
Input: X is the data content of the node to be inserted.

Output: A double linked list enriched with a node containing data X at the end of the list.

Data structure: Double linked list structure whose pointer to the header node is the *HEADER*.

Steps:

```
1.  ptr = HEADER
2.  While (ptr→RLINK ≠ NULL) do                    // Move to the last node
3.      ptr = ptr→RLINK
4.  EndWhile
5.  new = GetNode(NODE)                            // Avail a new node
6.  If (new ≠ NULL) then                           // If the node is available
7.      new→LLINK = ptr          // Change the pointer shown as 1 in Figure 3.11(b)
8.      ptr→RLINK = new          // Change the pointer shown as 2 in Figure 3.11(b)
9.      new→RLINK = NULL                           // Make the new node as the last node
10.     new→.DATA = X                              // Copy the data into the new node
11. Else
12.     Print "Unable to allocate memory: Insertion is not possible"
13. EndIf
14. Stop
```



(b) Inserting a node at the end

INSERTING A NODE INTO A DOUBLY LINKED LIST AT ANY POSITION IN THE LIST.

Algorithm InsertAny_DL

Input: X is the data content of the node to be inserted, and *KEY* the data content of the node after which the new node is to be inserted.

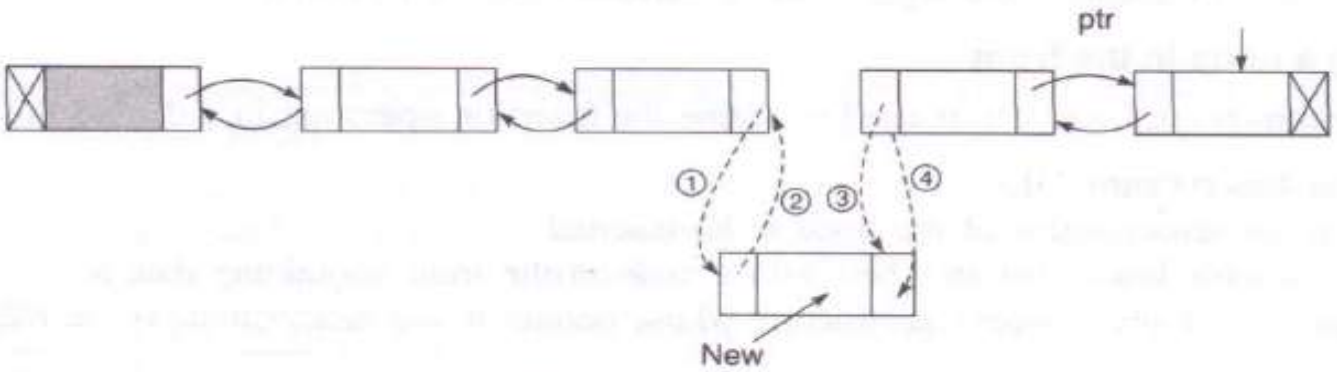
Output: A double linked list enriched with a node containing data X after the node with data *KEY*, if *KEY* is not present in the list then it is inserted at the end.

Data structure: Double linked list structure whose pointer to the header node is the *HEADER*.

Steps:

```
1.  ptr = HEADER
2.  While (ptr→DATA ≠ KEY) and (ptr→RLINK ≠ NULL)      // Move to the key node if the
                                  // current node is not the KEY node or if the list reaches the end
3.      ptr = ptr→RLINK
4.  EndWhile
5.  new = GetNode(NODE)                  // Get a new node from the pool of free storage
6.  If (new = NULL) then                 // When the memory is not available
7.      Print (Memory is not available)
8.      Exit                                                          // Quit the program
9.  EndIf
10. If (ptr→RLINK = NULL) then           // If the KEY is not found in the list
11.     new→LLINK = ptr
12.     ptr→RLINK = new                                               // Insert at the end
13.     new→RLINK = NULL
```

```
14.      new→DATA = X          // Copy the information to the newly inserted node
15.  Else                      // The KEY is available
16.      ptr1 = ptr→RLINK      // Next node after the key node
17.      new→LLINK = ptr       // Change the pointer shown as 2 in Figure 3.11(c)
18.      new→RLINK = ptr1      // Change the pointer shown as 4 in Figure 3.11(c)
19.      ptr→RLINK = new       // Change the pointer shown as 1 in Figure 3.11(c)
20.      ptr1→LLINK = new      // Change the pointer shown as 3 in Figure 3.11(c)
21.      ptr = new             // This becomes the current node
22.      new→DATA = X          // Copy the content to the newly inserted node
23.  EndIf
24.  Stop
```



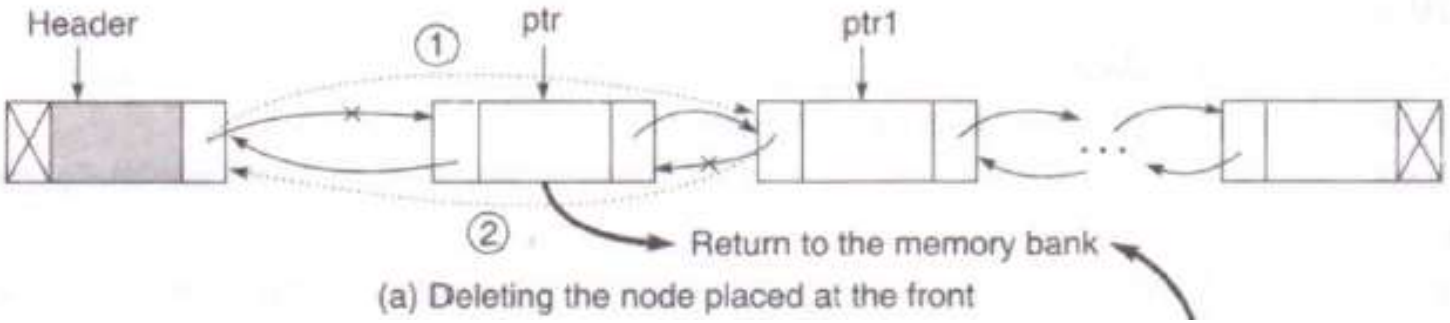
(c) Inserting a node at any intermediate position

DELETING THE NODE AT THE FRONT OF A DOUBLY LINKED LIST

Algorithm DeleteFront_DL
Input: A double linked list with data.
Output: A reduced double linked list.
Data structure: Double linked list structure whose pointer to the header node is the HEADER.

Steps:

```
1.  ptr = HEADER→RLINK                                   // Pointer to the first node
2.  If (ptr = NULL) then                                 // If the list is empty
3.      Print "List is empty: No deletion is made"
4.      Exit
5.  Else
6.      ptr1 = ptr→RLINK                                 // Pointer to the second node
7.      HEADER→RLINK = ptr1          // Change the pointer shown as 1 in Figure 3.12(a)
8.      If (ptr1 ≠ NULL)        // If the list contains a node after the first node of deletion
9.          ptr1→LLINK = HEADER      // Change the pointer shown as 2 in Figure 3.12(a)
10.     EndIf
11.     ReturnNode (ptr)                 // Return the deleted node to the memory bank
12. EndIf
13. Stop
```



(a) Deleting the node placed at the front

DELETING THE NODE AT THE END OF A DOUBLY LINKED LIST

Algorithm DeleteEnd_DL
Input: A double linked list with data.
Output: A reduced double linked list.
Data structure: Double linked list structure whose pointer to the header node is the *HEADER*.

Steps:

```
1.  ptr = HEADER
2.  While (ptr→RLINK ≠ NULL) do                    // Move to the last node
3.      ptr = ptr→RLINK
4.  EndWhile
5.  If (ptr = HEADER) then                         // If the list is empty
6.      Print "List is empty: No deletion is made"
7.      Exit                                       // Quit the program
8.  Else
9.      ptr1 = ptr→LLINK                           // Pointer to the last but one node
10.     ptr1→RLINK = NULL                          // Change the pointer shown as 1 in Figure 3.12(b)
11.     ReturnNode (ptr)                           // Return the deleted node to the memory bank
12. EndIf
13. Stop
```

(b) Deleting the node placed at the end

DELETING THE NODE FROM ANY POSITION OF A DOUBLY LINKED LIST

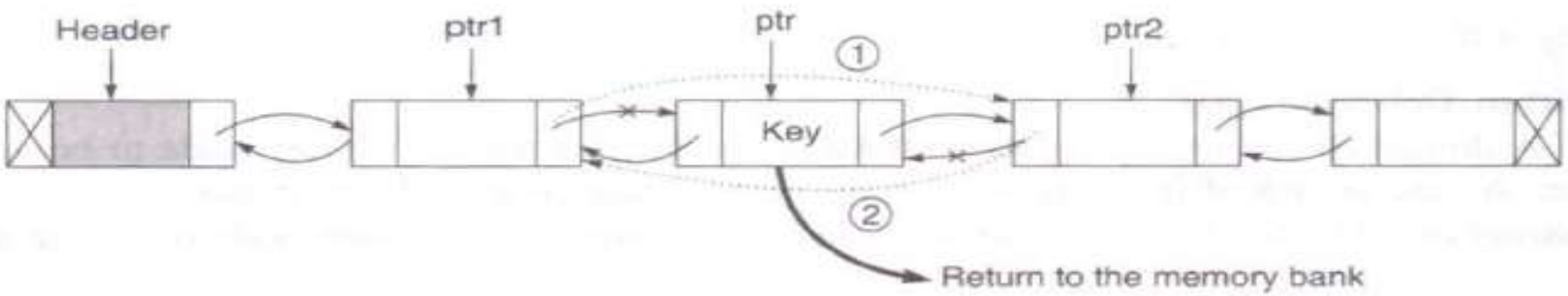
Algorithm DeleteAny_DL

Input: A double linked list with data, and *KEY*, the data content of the key node to be deleted.
Output: A double linked list without a node having data content *KEY*, if any.
Data structure: Double linked list structure whose pointer to the header node is the *HEADER*.

Steps:

```
1.  ptr = HEADER→RLINK                                   // Move to the first node
2.  If (ptr = NULL) then
3.      Print "List is empty: No deletion is made"
4.      Exit
5.  EndIf                                                // Quit the program
6.  While (ptr→DATA ≠ KEY) and (ptr→RLINK ≠ NULL) do     // Move to the desired node
7.      ptr = ptr→RLINK
8.  EndWhile
9.  If (ptr→DATA = KEY) then                             // If the node is found
10.     ptr1 = ptr→LLINK                                 // Track the predecessor node
```

```
11.      ptr2 = ptr→RLINK                          // Track the successor node
12.      ptr1→RLINK = ptr2                         // Change the pointer shown as 1 in Figure 3.12(c)
13.      If (ptr2 ≠ NULL) then                     // If the deleted node is the last node
14.          ptr2→LLINK = ptr1                     // Change the pointer shown as 2 in Figure 3.12(c)
15.      EndIf
16.      ReturnNode(ptr)                           // Return the free node to the memory bank
17.  Else
18.      Print "The node does not exist in the given list"
19.  EndIf
20.  Stop
```



(c) Deleting a node from any intermediate position

CIRCULAR LINKED LIST

CIRCULAR LINKED LIST

- **Circular linked list is a linked list** which consists of collection of nodes each of which **has two parts**, namely the **data part and** the **link part**.
- The **data part holds the value of the element** and the **link part** has the **address of the next node.**
- The **last node** of list has the **link pointing to the first node thus making** the **circular traversal possible in the list.**
- A linked list where the **last node points the header node** is called the **circular linked list**.

- The figure shows a pictorial representation of a circular linked list.

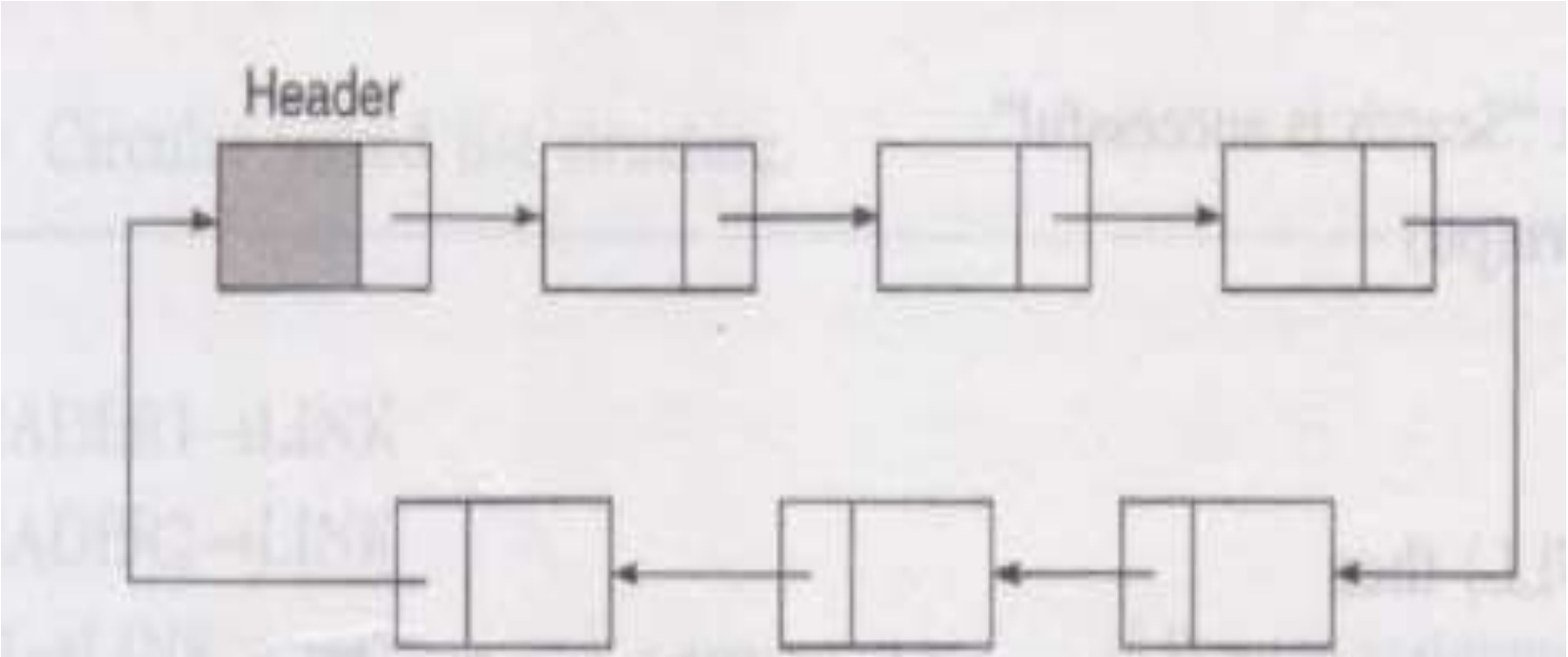


Figure 3.8 A circular linked list.

- Circular linked lists have certain advantages over ordinary linked list. Some advantages of circular linked lists are discussed below.

1. Accessibility of a member node in the list

In an ordinary list , a member node is accessible from a particular node, that is , **from the header node only.** But in a circular linked list, **every member node is accessible from any node** by merely chaining through the list.

2. NULL link problem.

The **NULL value** in the link field **may create some problem during the execution of problems if proper care is not taken.** We can solve this problem by using circular linked list.

STACK DATA STRUCTURE

STACK DATA STRUCTURE

- A stack is an Abstract Data Type (ADT), commonly used in most
- programming languages. It is named stack as it behaves like a real-world stack,
- for example – a deck of cards or a pile of plates, etc.

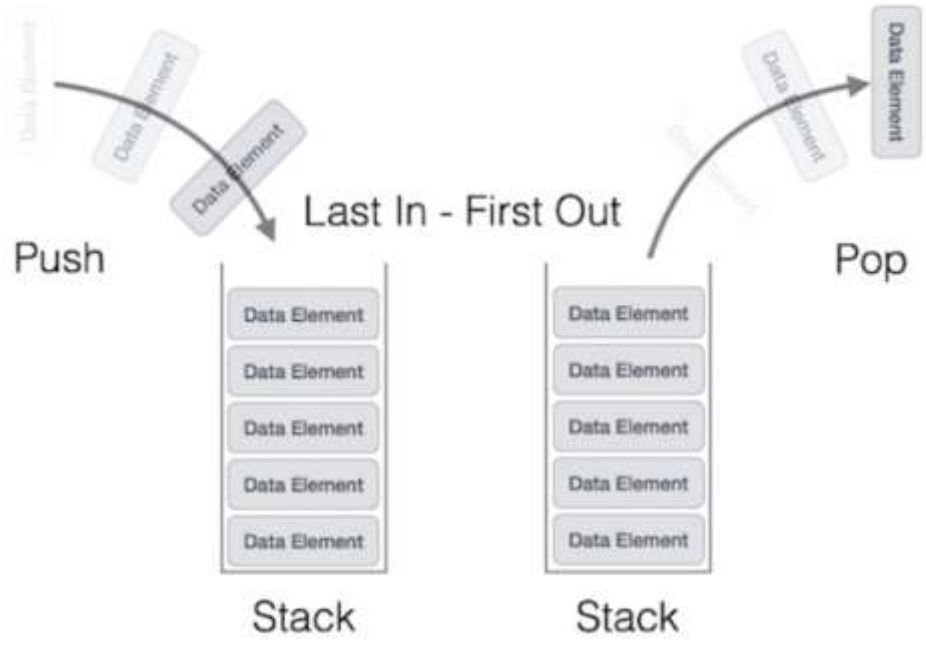
- A real-world stack allows operations at one end only. For example, we

can place or remove a card or plate from the top of the stack only. Likewise, Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack.

- This feature makes it LIFO data structure. LIFO stands for Last-in-first-out.
- Here, the element which is placed (inserted or added) last, is accessed first.
- In stack terminology, insertion operation is called PUSH operation and

removal operation is called POP operation.

STACK REPRESENTATION



A stack can be implemented by means of Array, Structure, Pointer, and Linked List. Stack can either be a fixed size one or it may have a sense of dynamic resizing.

BASIC OPERATIONS

- Stack operations may involve initializing the stack, using it and then de-
- initializing it. Apart from these basic stuffs, a stack is used for the following

two primary operations −

- push() − Pushing (storing) an element on the stack.
- pop() − Removing (accessing) an element from the stack.

OTHER FUNCTIONALITIES

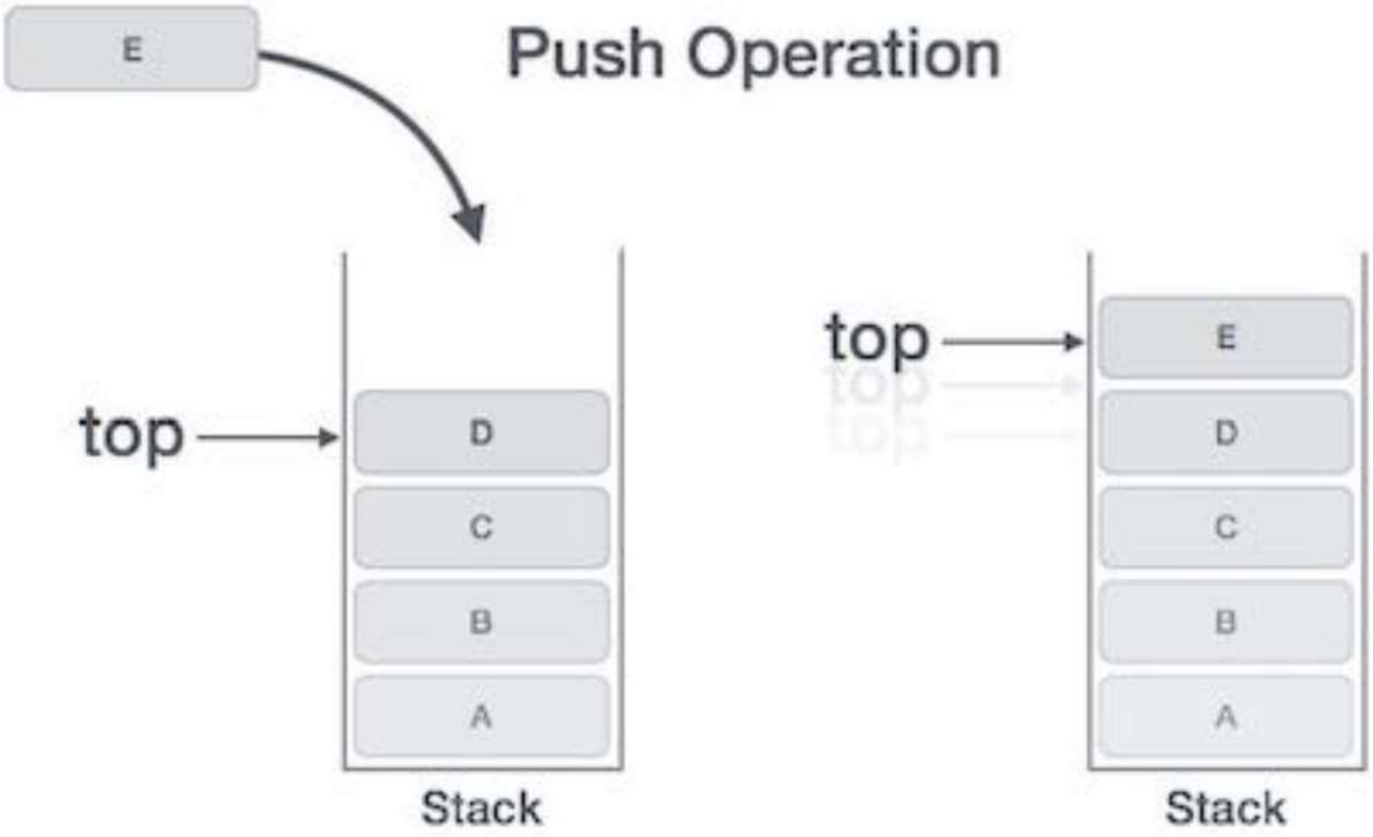
- peek() – get the top data element of the stack, without removing it.
- isFull() – check if stack is full.
- IsEmpty() - check if stack is empty.

At all times, we maintain a pointer to the last PUSHed data on the stack.
As this pointer always represents the top of the stack, hence named top.
The top pointer provides top value of the stack without actually removing it.

PUSH OPERATION

The process of putting a new data element onto stack is known as a Push

- Operation. Push operation involves a series of steps −
- Step 1 − Checks if the stack is full.
- Step 2 − If the stack is full, produces an error and exit.
- Step 3 − If the stack is not full, increments top to point next empty
- space.
- Step 4 − Adds data element to the stack location, where top is pointing.
- Step 5 − Returns success.


E
Push Operation
top
D
C
B
A
Stack
top
E
D
C
B
A
Stack

Algorithm for PUSH Operation

A simple algorithm for Push operation can be derived as follows −

```
begin procedure push: stack, data
   if stack is full
      return null
   endif
   top ← top + 1
   stack[top] ← data

end procedure
```

PROGRAM

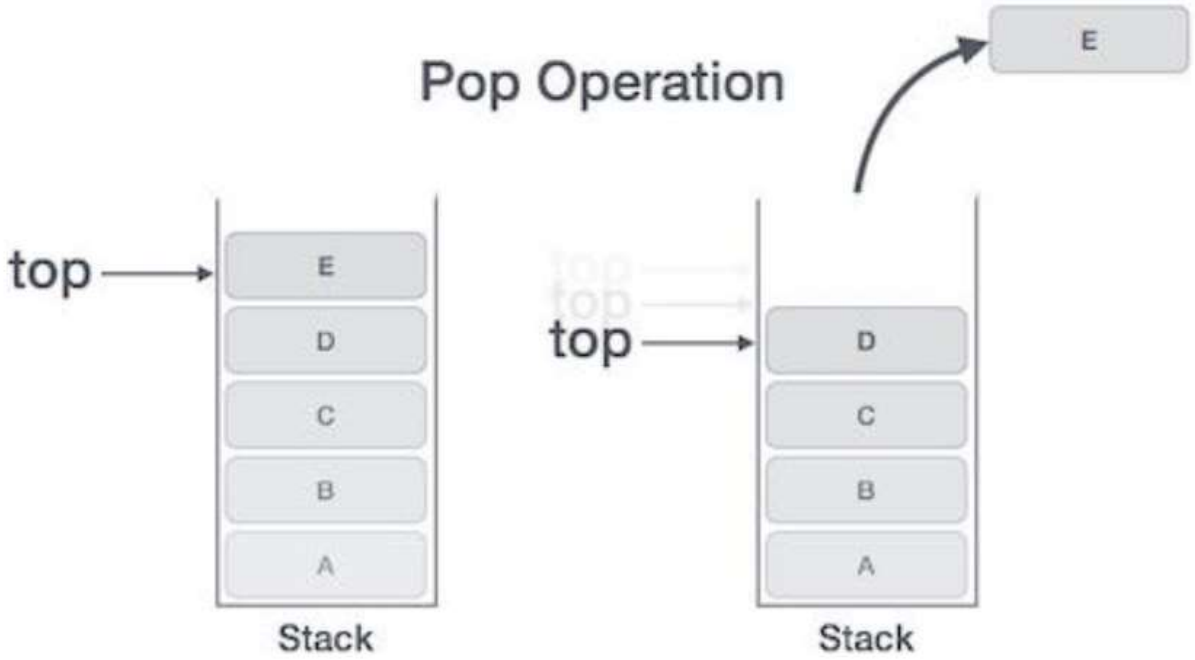
```
void push(int data)
{
  if(!isFull())
  {
    top = top + 1;
    stack[top] = data;
  }
  else
    printf("Could not insert data, Stack is full.\n");
  }
```

POP OPERATION

- Accessing the content while removing it from the stack, is known as a Pop Operation.
- In an array implementation of pop() operation, the data element is not actually removed, instead top is decremented to a lower position in the stack to point to the next value.
- But in linked-list implementation, pop() actually, removes data element and deallocates memory space.

STEPS

- Step 1 – Checks if the stack is empty.
- Step 2 – If the stack is empty, produces an error and exit.
- Step 3 – If the stack is not empty, accesses the data element at which top is pointing.
- Step 4 – Decreases the value of top by 1.
- Step 5 – Returns success.


Pop Operation
E
top
E
D
C
B
A
Stack
top
D
C
B
A
Stack

ALGORITHM FOR POP OPERATION

- begin procedure pop: stack
- if stack is empty
- return null
- endif
- data ← stack[top]
- top ← top - 1
- return data
- end procedure

PROGRAM

```
int pop(int data)
{
  if(!isempty())
  {
     data = stack[top];
     top = top - 1;
     return data;
  } else
     printf("Could not retrieve data, Stack is empty.\n");
}
```